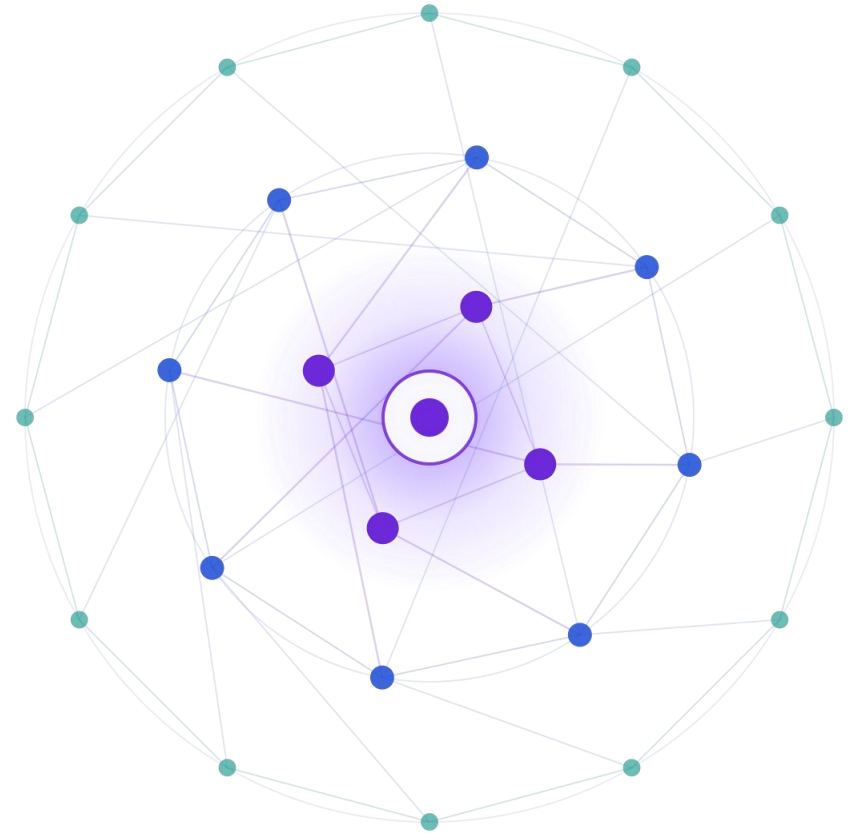


SAMSUNG PRISM

Generative AI Hackathon

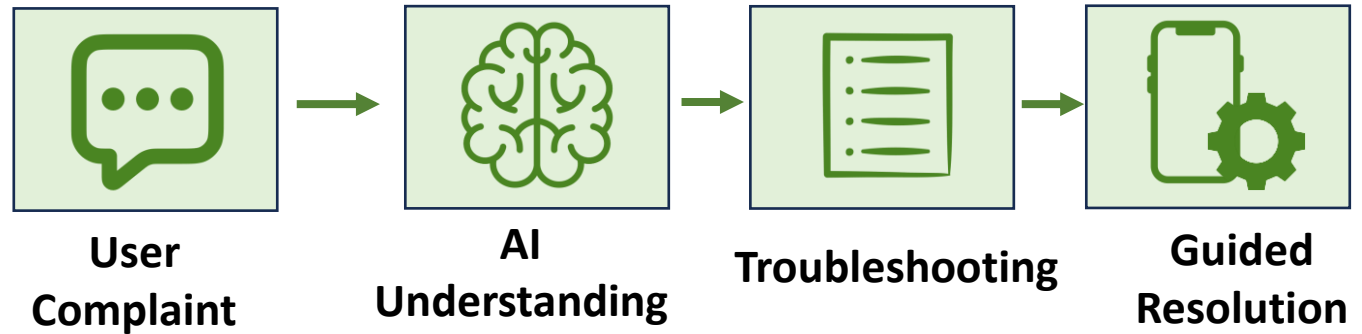
3rd Edition 2026 – 27

- Theme ID - 2
- Team Name - Hustle
- College Name – MS Ramaiah Institute of Technology
- Member Name & Email 1- Nidhi C A & 1ms24cs110@msrit.edu
- Submission Github link - <https://github.com/nidhica/SGTE-Smart-Guided-Troubleshooting-Engine>



Smart Guided Troubleshooting Engine

AI-Assisted Device Troubleshooting
and Guided Resolution



Existing Solutions & Gaps



Existing solutions



Traditional Customer Support:

Users contact support agents who manually diagnose issues and suggest solutions. This often involves multiple interactions to identify the problem.



Knowledge Base Systems:

Provide troubleshooting articles and step-by-step instructions for common device problems. Users must search for relevant articles themselves.



Automated Troubleshooting Tools:

Offer predefined diagnostic flows for specific device issues, such as display problems or touchscreen failures.



Online Communities & Forums:

Users seek solutions from other users who have experienced similar issues, but the advice may vary in accuracy and relevance.



Device Diagnostic Tools:

Help identify certain hardware or software problems through built-in tests and diagnostic features.



Key Gaps



Limited Understanding:

Difficulty interpreting vague, informal, and complex user complaints.



Manual Effort:

Users must search through multiple articles and navigate device settings themselves.



Lack of Personalization:

Generic solutions may not address the user's specific symptoms or device context.



Limited Actionability:

Instructions may lack direct navigation links to relevant device settings.



Insufficient Grounding:

Recommendations may not always be supported by verified troubleshooting information.



Fragmented Troubleshooting:

Relevant information may be spread across different support articles and resources.



Difficulty Handling Complex Symptoms:

Multiple symptoms occurring together can make it harder to identify the appropriate troubleshooting path.

Our Solutions & Architecture Diagram

Solution

SGTE is an AI-assisted troubleshooting platform that understands user complaints, generates structured troubleshooting plans and provides relevant navigation links.



Understand

Accepts natural-language complaints
Normalizes informal queries into technical format



ANALYZE

Identifies troubleshooting goals.
Extracts relevant actions and generates UI steps.



NAVIGATE

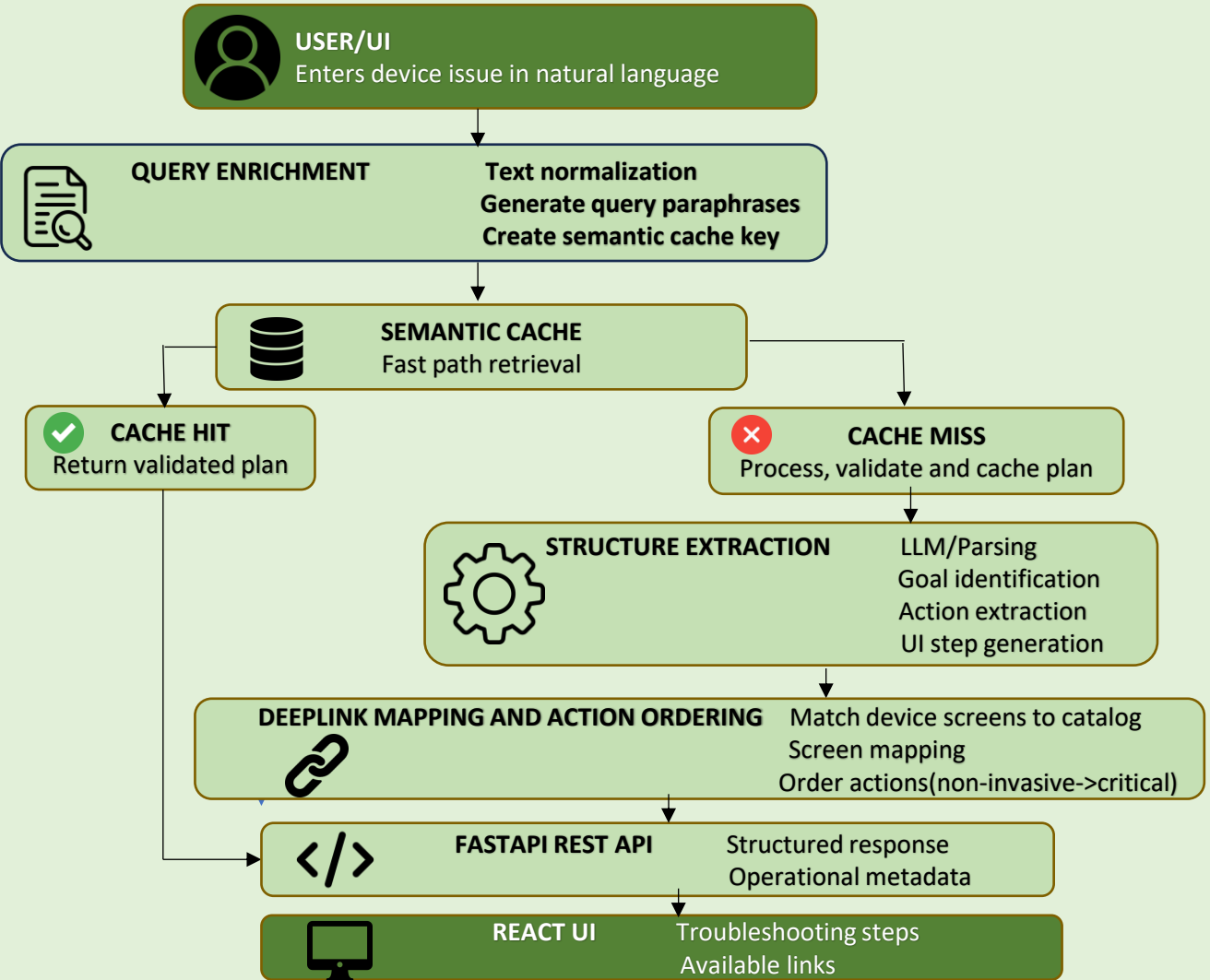
Maps device screens to available deep links.
Organizes actions from non-invasive to critical steps



OPTIMIZE

Uses semantic caching to reuse validated plans.
Reduces repeated LLM processing and improves response time.

Architecture Diagram



Demo & Product Walkthrough



Enter the Device Complaint

Natural Language input

The user describes the device issue in everyday language

SGTE

Smart-Guided Troubleshooting Engine

About

SGTE | TROUBLESHOOTING STUDIO

Something wrong with your device?

Get clear, grounded troubleshooting steps for your Samsung device. SGTE uses verified Samsung support evidence and provides exact Settings links when available.

My phone screen is blank

24/2000

Start Diagnosis →

QUICK-SELECT CATEGORIES

Battery & Charging

Display & Screen

Touch & Gestures

Connectivity

Categories fill a supported example complaint — they do not invent a troubleshooting result. Choose Start Diagnosis to run the grounded plan.

Have troubleshooting context?

HOW SGTE WORKS

Describe → Diagnose → Resolve



Process the Complaint

Intelligent Query Processing

SGTE understands the query, extracts key information and generates a structured plan.



Building your troubleshooting plan...

Working through the diagnosis pipeline. Stage highlights are illustrative while the request is in flight — not live progress percentages.

- 1 Understanding issue**
Analyzing your query and identifying key symptoms... In progress
- 2 Finding relevant evidence**
Searching knowledge base and support articles... Waiting
- 3 Building troubleshooting plan**
Generating step-by-step actions... Waiting
- 4 Checking actions**
Validating actions and device compatibility... Waiting
- 5 Resolving Settings actions**
Mapping navigation links to relevant device screens... Waiting



This usually takes a few seconds

We're analyzing your complaint and preparing a clear, structured plan.



Get Troubleshooting Guidance

Structured troubleshooting plan

The system generates a clear, step by step plan with actions and explanations

SGTE

Smart-Guided Troubleshooting Engine

New session

About

DIAGNOSTIC REPORT

MOCK

SGTE Insights

Black screen

ORIGINAL USER QUERY

"My phone screen is blank"

ISSUE

Black screen

ACTIONS

1 issue - 3 steps

CATALOGUE LINKS

None on this plan

Fast path

Response served from cache - 16 ms - MOCK

RECOMMENDED ACTIONS

01

CRITICAL

Force a Restart

It will force a restart

- Press and hold both the Power button (or Side button) and the Volume down button simultaneously for at least 20 seconds.

Manual-only action

No verified catalogue deeplink — follow the guidance manually. No valid catalogue URL is attached for this action.

02

MANUAL

Check for Physical Damage and Liquid Exposure

It will check for physical damage

Tools and tech stack used

1 Frontend User Interface  React.js Vite Builds a responsive and interactive interface for users to enter device complaints and view troubleshooting guidance.	2 Backend API and Processing  Python FastAPI Pydantic Implements the troubleshooting pipeline, handles requests, validates structured Data and returns responses.	3 API and Query Processing Understanding User Input  LLM Provider NLP Query Enrichment Converts informal complaints into structured queries and generates a step-by-step troubleshooting plan.	4 Retrieval and Knowledge  Dense Retrieval TF-IDF Knowledge Base SIIS Retrieves relevant troubleshooting information and maps device navigation destinations using the deep-link catalog.	5 Development and Testing  Pytest Git Github Tests backend functionality, tracks code changes, and manages the project repository.
--	--	--	---	--

Impact & Use case

Impact



Simplified Troubleshooting

Transforms everyday device complaints into clear, structured troubleshooting steps.



Reduced User Effort

Reduces the need to search through multiple support articles and manually locate settings.



Actionable Guidance

Presents step-by-step instructions and available navigation links to help users troubleshoot device issues.



Faster Repeat Responses

Uses semantic caching to reuse validated troubleshooting plans for matching queries, reducing repeated processing.



Improved Accessibility

Makes technical troubleshooting instructions easier to understand for users with different levels of technical knowledge.

Real-World Use Case



Display and Screen Issues

A user reports screen flickering, dimness, or a black screen.



Connectivity Problems

A user experiences Wi-Fi disconnections or Bluetooth connection issues.



Device Settings Assistance

A user needs help finding the correct display, sound or accessibility settings.



Application Issues

A user reports an application freezing or behaving unexpectedly.



Device Functionality Issues

A user experiences problems with notifications, touch response or other device features.

Innovation highlights, results and limitations



Innovation Highlights

Key features that make SGTE unique



Natural Language Understanding

Processes everyday device complaints and transforms informal queries into structured technical inputs.



Hybrid Retrieval

Combines dense retrieval with TF-IDF and lexical matching to identify potentially relevant troubleshooting information.



Structured Troubleshooting

Extracts troubleshooting goals, actions and individual UI steps to generate organized guidance.



Semantic Caching

Reuses validated troubleshooting plans for matching queries, reducing repeated processing.



Guided Navigation

Maps relevant device screens to available deep-link catalog entries and organizes troubleshooting actions.

 Result	
Metric	Result
Scenario coverage	22 predefined troubleshooting scenarios
Phase 79 tests	73 tests passed
Full test suite	885 tests passed
Fabricated evidence IDs in scenario overrides	0
Deep links in scenario override responses	0
 Key Achievement Developed a functional troubleshooting prototype with query processing, structured guidance, retrieval components and a web interface.	



Limitations

1

Limited Scenario Coverage

The manually authored scenarios cover a finite set of specific device complaints.

2

Retrieval Relevance

The relevance of retrieved information can vary, particularly for complex or less common symptoms.

3

Incomplete Navigation Support

Verified deep links are not available for every troubleshooting action.

4

Generic Guidance

Some complex or multi-symptom complaints may receive broad or less-targeted recommendations.

5

Production Readiness

The current prototype uses mock-mode configuration and has not been certified for production use.

What's next

1

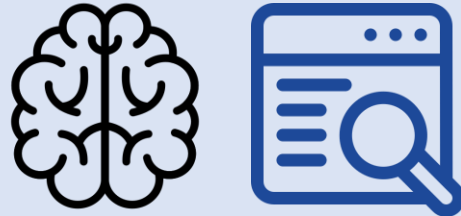
Expand Troubleshooting Coverage



- Support a wider range of device issues and user complaints.
- Add more device-specific troubleshooting scenarios.

2

Improve AI and Retrieval Accuracy



- Enhance query understanding for complex, multi-symptom complaints.
- Improve the relevance of retrieved troubleshooting information.

3

Strengthen Evidence and Navigation



- Integrate more verified troubleshooting resources.
- Expand verified deep-link coverage for device settings.

4

Enable Production Readiness



- Integrate and validate a live AI provider.
- Improve reliability, response consistency and error handling.

5

Evaluate and Scale



- Evaluate troubleshooting accuracy using a representative test dataset.
- Expand support to additional devices and languages.

Brownie points slide(differentiation)

1 Natural-Language First

Users describe the problem in their own words instead of selecting from rigid diagnostic menus.



"My screen keeps flickering and then goes black."

2

From Complaint to Structured Plan

Transforms the user's description into an organized troubleshooting workflow.



Complaint



Goal



Actions



UI Steps

3

Actionable Device Navigation



Step:
Open Display
Settings

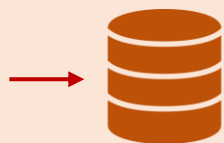
Troubleshooting guidance can be connected to relevant device screens through the deep-link catalog, reducing the need for users to manually search through settings.

4

Semantic Cache for Faster Repeated Queries

Previously processed queries can use validated cached plans, avoiding unnecessary repeated processing when an appropriate cached result exists.

"Wi-Fi keeps disconnecting"



Semantic Cache

(previously processed queries)



Return validated
path

5

Controlled and Structured Responses

The pipeline applies response structure, schema validation, action ordering, and output constraints to keep troubleshooting guidance consistent and controlled.



Response
Structure



Schema
Validation



Action
Ordering



Output
Constraints

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y/N) - Y
- README with reproducible setup instructions (Y/N) - Y
- Demo video, max 5 minutes (YouTube or Drive link) - Y
- Presentation file (PPT or PDF) (Y/N) - Y

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India